Muhammad Saad Khan (251309919)

1.

Fn = Fn−1 + Fn−2, n > 1; F1 = 1; F0 = 0.

Let A = (1 + √5)/2. Using the equation $A^2 = A + 1$, prove that

A^(n−2) ≤ Fn ≤ A^n, for n ≥ 1.

proof:

base cases:

n = 1: F1 = 1 and A^(-1) ≤ 1 ≤ A

n = 2: F2 = 1 and A^0 = 1 ≤ 1 ≤ A^2

assume A^(n−2) ≤ Fn ≤ A^n and A^(n−3) ≤ F(n−1) ≤ A^(n−1)

inductive step:

Fn+1 = Fn + F(n−1)

upper bound:

Fn+1 ≤ A^n + A^(n−1)

= A^(n−1)(A + 1)

= $A^{n-1}A^2$

= A^(n+1)

lower bound:

Fn+1 ≥ A^(n−2) + A^(n−3)

= A^(n−3)(A + 1)

= A^(n−3)A²

= A^(n−1)

A^(n−1) ≤ Fn+1 ≤ A^(n+1), completing the induction

b)

A^(n−2) ≤ Fn ≤ A^n,

(n−2)log2(A) ≤ log2(Fn) ≤ nlog2(A)

log2(A) ≈ 0.694:

F50:

33.3 ≤ log2(F50) ≤ 34.7

F50 needs about 50 bits

F500:

345.6 ≤ log2(F500) ≤ 347.0

And F500 needs 348

2)

a) worst case = Θ(k²)

total time = (n/k) · Θ(k²) = Θ(nk)

b) n/k sorted sublists each of length k

each merge accounts for all n elements

number of levels = log(n/k)

total merge time = Θ(n log(n/k))

c) total run time = Θ(nk + n log(n/k))

standard merge sort time = Θ(n log n)

Θ(n log n) = nk + n log(n/k)

Θ(log n) = k + log(n/k)

Largest k which is allowed is k = Θ(log n)

d) since insertion sort is fast for small arrays k should be chosen as a small value to improve on this characteristic.

3)

Case	O	o	Ω	ω	Θ
A)	yes	yes	no	no	no
b)	yes	yes	no	no	no
c)	no	no	no	no	no
D)	no	no	yes	yes	no
e)	yes	no	yes	no	yes
f)	yes	no	yes	no	yes

4)

T(n) <= T(n/2) + c

<= T(n/4) + c + c

<= T(n/8) + c + c + c

Stops at (n/2^k) = 2

$n = 2^{(k+1)}$

$k = \log_2 n - 1$

depth = $\Theta(\log n) = T(n)$

proof:

$T(n) = O(\log n)$

base case:

$n = 2$

$T(2) \leq c$

Choose $a \geq c/\log 2$

$T(2) \leq a \log 2$

Inductive step

$T(n) \leq T(n/2) + c$

$\leq a \log (n/2) + c$

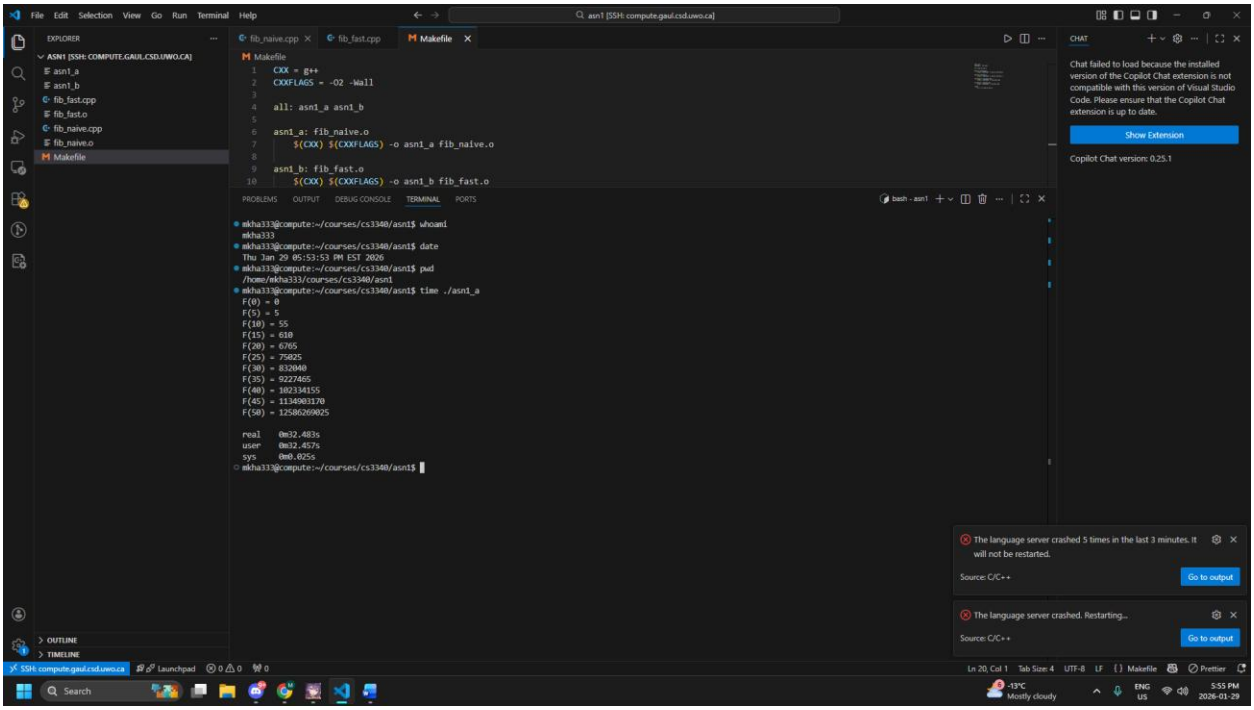
$= a \log (n-1) + c$

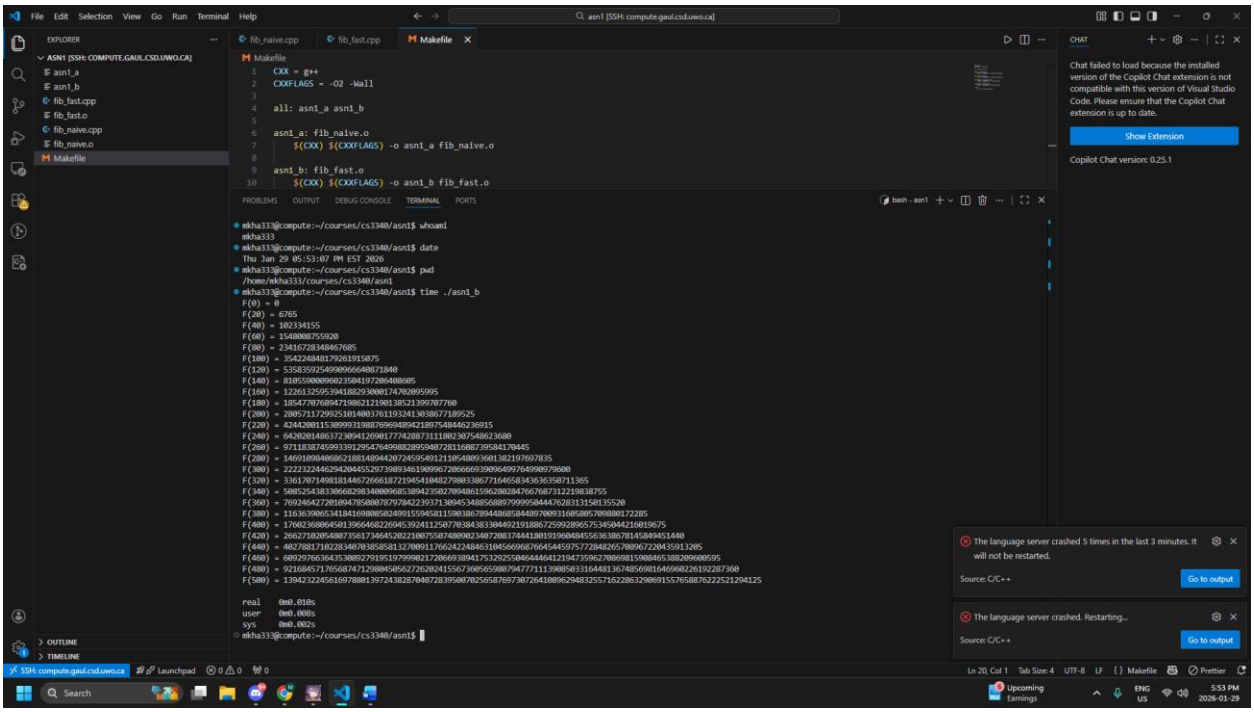
$= a \log n - a + c$

$T(n) \leq a \log n$

5)

c)





The naïve approach has an exponential time complexity which leads to the algorithm becoming really slow as n increases due to redundant computations. While the optimized version has a linear time complexity which makes it efficient to compute high Fibonacci numbers.

d) 5 a cannot compute f50 using 4 byte int since f50 would result in an overflow. 5b is able to computes f500 so precisely since it uses a custom big integer and stores all digits explicitly

5a algorithm:

```
long long fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    for (int i = 0; i <= 10; i++) {
        int n = i * 5;
        cout << "F(" << n << ") = " << fib(n) << endl;
    }
    return 0;
}
```

Time complexity = O(n^2)

5b algorithm:

```
struct BigInt {
    vector<int> d;

    BigInt(long long x = 0) {
        if (x == 0) d.push_back(0);
```

```
        while (x > 0) {
            d.push_back(x % 10);
            x /= 10;
        }
    }

    BigInt operator+(const BigInt& other) const {
        BigInt res;
        res.d.clear();
        int carry = 0;
        size_t n = max(d.size(), other.d.size());
        for (size_t i = 0; i < n || carry; i++) {
            int sum = carry;
            if (i < d.size()) sum += d[i];
            if (i < other.d.size()) sum += other.d[i];
            res.d.push_back(sum % 10);
            carry = sum / 10;
        }
        return res;
    }

    void print() const {
        for (int i = d.size() - 1; i >= 0; i--)
            cout << d[i];
    }
};
```

```
pair<BigInt, BigInt> fib_fast(int n) {
    if (n == 0) return {BigInt(0), BigInt(0)};
    if (n == 1) return {BigInt(1), BigInt(0)};
    auto prev = fib_fast(n - 1);
    BigInt Fn = prev.first + prev.second;
    return {Fn, prev.first};
}

int main() {
    for (int i = 0; i <= 25; i++) {
        int n = i * 20;
        auto result = fib_fast(n);
        cout << "F(" << n << ") = ";
        result.first.print();
        cout << endl;
    }
    return 0;
}
```

Time complexity = O(n)